

Lab 3 — The Command Line, and git

How you'll submit everything for the rest of the semester

Amir Farbin

Today

I'm not here — read this first

- ▶ **Today's session is a posted video.** No live Q&A; bring questions to Lab 4
- ▶ **Nothing to turn in, and nothing to set up.** Today is background
- ▶ Lab 3's exercises use the Lecture 6–7 material — probability, histograms, Monte Carlo

I'm not here — read this first

- ▶ **Today's session is a posted video.** No live Q&A; bring questions to Lab 4
- ▶ **Nothing to turn in, and nothing to set up.** Today is background
- ▶ Lab 3's exercises use the Lecture 6–7 material — probability, histograms, Monte Carlo

Due dates

- ▶ **Lab 2 — Friday 9/11**, submitted *in* the Lab 4 session
- ▶ **Lab 3 — Friday 9/18**

What the video covers, and where it's now different

The video is from an earlier offering. Everything it says about the **command line** still stands. Four things have moved on:

- ▶ It says “*next week I’ll talk to you about git*” — that material is now **written down**, in this deck and in the setup guide, rather than lectured
- ▶ **Authentication is new.** GitHub passwords stopped working; the video predates that. Use `gh auth login`
- ▶ The repository is `DATA3402.Fall.2026`. Any other name you hear or read is from a past semester
- ▶ It notes a date on the slide that hadn’t been updated. Ignore dates in the video — the ones on the previous slide are correct

What I'm asking of you this week

Just this: get comfortable enough on the command line to move around and copy files.

What I'm asking of you this week

Just this: get comfortable enough on the command line to move around and copy files.

I am not asking you to set up GitHub today. We do that together in Lab 4, on Friday 9/11, from scratch. **You will not be behind if you arrive having done nothing.**

...and if you want to look ahead

The rest of this deck is git and submission. **I am not going over it this week — I'll go through it with you next week.** It's here for anyone who wants to read ahead.

...and if you want to look ahead

The rest of this deck is git and submission. **I am not going over it this week — I'll go through it with you next week.** It's here for anyone who wants to read ahead.

If you'd like to try something early, try **GitHub authentication**:

- ▶ Instructions are in the **lab notebook**, Labs/Lab.3/Lab.3.ipynb
- ▶ It's the piece most likely to go wrong on an individual machine, so having it working makes next week quick for you

...and if you want to look ahead

The rest of this deck is git and submission. **I am not going over it this week — I'll go through it with you next week.** It's here for anyone who wants to read ahead.

If you'd like to try something early, try **GitHub authentication**:

- ▶ Instructions are in the **lab notebook**, Labs/Lab.3/Lab.3.ipynb
- ▶ It's the piece most likely to go wrong on an individual machine, so having it working makes next week quick for you

If you try it and it doesn't work, stop and bring it to Lab 4. Debugging together is what that session is for. Don't lose an evening to it alone.

The command line

Why bother

- ▶ You've interacted with computers through **graphical metaphors** — icons, windows, drag and drop. They're easy because they hide the machine
- ▶ Development and data work need you **closer to how the machine actually works**
- ▶ Some things simply cannot be done through a GUI

Why bother

- ▶ You've interacted with computers through **graphical metaphors** — icons, windows, drag and drop. They're easy because they hide the machine
- ▶ Development and data work need you **closer to how the machine actually works**
- ▶ Some things simply cannot be done through a GUI
- ▶ And yes, you can ask an AI for the command — but if you don't understand the answer, you won't know whether it did the right thing, or how to fix it when it doesn't

The shell

- ▶ Without a GUI, the interface to a machine is a **command-line shell**
- ▶ It shows a **prompt**, which tells you where in the filesystem you are
- ▶ You type a command, press enter, it runs

The shell

- ▶ Without a GUI, the interface to a machine is a **command-line shell**
- ▶ It shows a **prompt**, which tells you where in the filesystem you are
- ▶ You type a command, press enter, it runs
- ▶ Mostly the shell just **runs other programs** — you type the program's name
 - ▶ so it has to know *where to look*: that's the **PATH** environment variable
- ▶ In Unix almost everything is a program: `ls`, `rm`, `mkdir`, `rmdir`, ...
- ▶ A few things are built into the shell itself, like `cd`
- ▶ **Environment variables** are a map of name → value that programs read to find things

Moving around

- ▶ `pwd` — print working directory: where am I?
- ▶ `ls` — list what's here · `ls -l` — with sizes, dates, permissions
- ▶ `cd <dir>` — go in · `cd ..` — go up one · `cd ../..` — up two · `cd -` — back where I was

Moving around

- ▶ `pwd` — print working directory: where am I?
- ▶ `ls` — list what's here · `ls -l` — with sizes, dates, permissions
- ▶ `cd <dir>` — go in · `cd ..` — go up one · `cd ../..` — up two · `cd -` — back where I was

Files

- ▶ `mkdir <dir>` — make a directory · `rmdir <dir>` — remove an empty one
- ▶ `cp <from> <to>` — copy · `mv <from> <to>` — move or rename · `rm <file>` — delete
- ▶ `man <command>` — the manual. Space to scroll, q to quit

This is the same navigating you do in Finder or Explorer — just typed.

How you'll actually write code

- ▶ **Most basic:** a text editor for the code, comments for the explanation, a shell to run it
- ▶ **IDEs** put editing, running and debugging in one place
- ▶ **Notebooks** combine text, code and output in one document — which is why this course lives in them

git — *background reading*

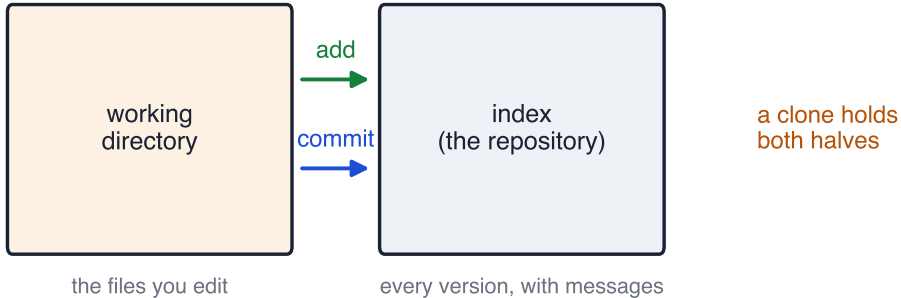
What git is

- ▶ A **version control system** — it tracks how files change over time
- ▶ Built for code, good for any large document
- ▶ Lets several people work on one thing without overwriting each other
- ▶ Lets you keep parallel versions, and mark releases
- ▶ Works entirely on your own machine, or against a server

What git is

- ▶ A **version control system** — it tracks how files change over time
- ▶ Built for code, good for any large document
- ▶ Lets several people work on one thing without overwriting each other
- ▶ Lets you keep parallel versions, and mark releases
- ▶ Works entirely on your own machine, or against a server
- ▶ **GitHub** runs a public git server, and has become how open-source code is shared
- ▶ **GitLab** is a similar thing companies often host themselves

A repository holds two things



- ▶ **add** puts a file's changes in line to be recorded
- ▶ **commit** records them, with a message saying what you did

The words you'll see

- ▶ **Clone** — your own local copy of a repository. What you actually work in
- ▶ **Remote** — a copy living on a server, that your clone talks to
- ▶ **Fork** — a copy on *your* GitHub account that can go its own way
- ▶ **Branch** — a parallel version that doesn't disturb the main one (`main`)
- ▶ **Tag** — a name pinned to one particular version

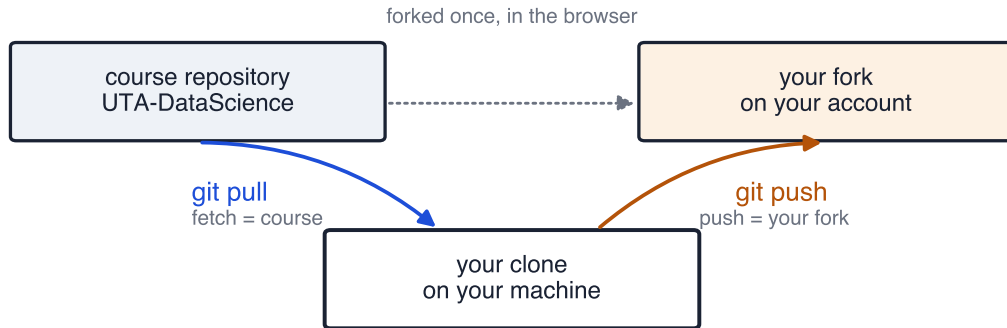
The words you'll see

- ▶ **Clone** — your own local copy of a repository. What you actually work in
- ▶ **Remote** — a copy living on a server, that your clone talks to
- ▶ **Fork** — a copy on *your* GitHub account that can go its own way
- ▶ **Branch** — a parallel version that doesn't disturb the main one (`main`)
- ▶ **Tag** — a name pinned to one particular version

- ▶ **fetch** — get changes from a remote · **merge** — fold them into your files
- ▶ **pull** = fetch + merge · **push** — send your commits to a remote
- ▶ **Merge conflict** — when two people changed the same thing and git can't decide.
Whoever pushes second resolves it

Submitting your work — *next week, not today*

How this course is wired



one remote, named `origin` — it fetches from one place and pushes to another

Pull from class. Push to my fork. Never force push.

The actual commands are in Lab 4

Wiring this up takes about six commands. **They're in** `Labs/Lab.4/Lab.4.ipynb`, which is released now, and we run them together on **Friday 9/11**.

The actual commands are in Lab 4

Wiring this up takes about six commands. **They're in Labs/Lab.4/Lab.4.ipynb**, which is released now, and we run them together on **Friday 9/11**.

They used to be in the Lab 3 notebook as well. Having them in two places was confusing, so there is now one copy, in the session where we actually do it.

- ▶ Labs/Lab.4/Lab.4.ipynb — the steps, in order
- ▶ Labs/GitHub-Setup.pdf — the same ground in more detail, with an error table

Every week

every lab, the same five steps



Rename your solution. If we hand you `Lab.3.ipynb`, work in `Lab.3.solution.ipynb` — otherwise the next `git pull` tries to merge our changes into yours, and a merge conflict inside a notebook lands in JSON rather than in your code.

Where to look

This week

- ▶ Labs/Lab.3/Lab.3.ipynb — the command-line reference, the **GitHub authentication instructions**, and the exercises due 9/18

Where to look

This week

- ▶ Labs/Lab.3/Lab.3.ipynb — the command-line reference, the **GitHub authentication instructions**, and the exercises due 9/18

Next week, or now if you want to read ahead

- ▶ Labs/Lab.4/Lab.4.ipynb — the submission setup, step by step
- ▶ Labs/GitHub-Setup.pdf — the same in more detail, with an error table
- ▶ Labs/DATA3402_Lab3_Merge_Conflict_Guide.pdf — if a notebook conflict happens

Where to look

This week

- ▶ Labs/Lab.3/Lab.3.ipynb — the command-line reference, the **GitHub authentication instructions**, and the exercises due 9/18

Next week, or now if you want to read ahead

- ▶ Labs/Lab.4/Lab.4.ipynb — the submission setup, step by step
- ▶ Labs/GitHub-Setup.pdf — the same in more detail, with an error table
- ▶ Labs/DATA3402_Lab3_Merge_Conflict_Guide.pdf — if a notebook conflict happens

Pull from class. Push to my fork. Never force push.